Functions and arguments

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

Teach what executes and what produces output.

Supply complete definitions/calls, expose ignored-argument bugs and distinguish arithmetic from the capacity contract.

Keep one habit: Ask when the body runs, which value each parameter receives and which statement displays a result.

MAIN LESSON · SECTION 1

Prepare the connected bridge lesson

> Check prior exposure and keep execution practice distinct from paper reasoning.

Goal and main plan

Primary target: Distinguish a **function definition** from a **function call**

Session · Sequence and minutes · Resources

> **Session:** Session 1 · Understand · 45 min

> **Sequence and minutes:** 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.
>
> **Resources:** Sections 1–6; record the brief recall in the workbook.

> **Session:** Session 2 · Practise · 45 min
>
> **Sequence and minutes:** 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.
>
> **Resources:** Sections 7–10; use the matching workbook W2 and misconception check.

> **Session:** Session 3 · Investigate and transfer · 45 min
>
> **Sequence and minutes:** 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.
>
> **Resources:** Use “Run the investigation”, “Connect your project”, “Show what you know”, “Study a complete case” and “Try a new case independently”. Give extra time for the full task set. If this week has a coding lab, use its named section; section numbers can change when extra explanations are included.

This is a starting allocation of 135 minutes, not a validated duration. Preserve all named topics. Schedule extra time for connected examples, the full investigation, independent cases and project building.

Repeated copies of a case share one recorded attempt; they are not new **evidence**.

Case exposure check: this source example appears in ai-2/2, python-bridge/8. Reuse a familiar case for re**call** or modelling; verify that the case used to assess transfer has not already received feedback.

Retrieval and prerequisite guidance

Lists and indexing

Without opening the old lesson, explain one key idea from Lists and indexing. Give an example and a mistake that the idea helps you avoid.

Look for: Distinguish a list index from the value stored at that position

Reference example, not the only acceptable wording: Length 3; indices 0, 1, 2; counts[1] is 7. The equal values occupy different positions, indices 0 and 2.

Branching and boundary tests

Without opening the old lesson, explain one key idea from Branching and boundary tests. Give an example and a mistake that the idea helps you avoid.

Look for: Trace the selected branch in an if elif else chain

Reference example, not the only acceptable wording: if tests negative first; elif checks <= 12 only after that fails; else handles the rest.

Unindented print runs after any selected branch, whose assignment gives label a value.

Values variables and types

Without opening the old lesson, explain one key idea from Values variables and types. Give an example and a mistake that the idea helps you avoid.

Look for: Distinguish integer, floating-point, text and Boolean values

Reference example, not the only acceptable wording: 12 is int; 1.5 is float; "12" is str; False is bool; "False" is str. Quotes mark text even when the characters spell a Boolean name.

Teach **definition**, **call**, **parameter** and positional **argument** as distinct ideas. Keep **input**s scalar and avoid mutable-list aliasing in this first **function** lesson.

Reference: https://docs.python.org/3/tutorial/controlflow.html#defining-**function**s. These **function**s print results; reusable return values and local state are taught in Week 9.

Materials and readiness

Python 3 editor and **function**s.py

Parameter cards labelled capacity and used

Call cards and a table for expected and observed **output**

8. Subtraction is ordered: reversing these values would produce -8.

End goal

Trace **function**s and **argument**s and explain the changed program before running it.

▼ Earlier wording, alternatives and picture transcripts

Teach the learning cycle

Retrieval answer guidance

Outcomes

Distinguish a **function definition** from a **function call**

Match positional **argument**s to **parameter**s in order

Reuse one **function** and test that it uses the supplied values

Preparation

Prerequisite answer

End goal for the pictorial case

This added worked case illustrates the week's main idea. Retain all original and connected topics; schedule its practice within the teaching cycle.

MAIN LESSON · SECTION 2

Teach function events and argument order explicitly

> Preview return sufficiently before the selected transfer uses it.

Read remaining-space requirement and assumptions

A fictional supply cupboard records capacity and the number of occupied spaces. The remaining-space calculation is capacity minus used.

Instead of copying the same instructions for every cupboard, define one **function** and **call** it with different values. A **function** makes a procedure reusable; its name alone does not guarantee that its instructions are correct.

The main remaining-space example assumes nonnegative integer counts with used<=capacity; it does not validate them. Its **function** prints a result. A short return-value preview below prepares you to read the selected transfer script: return hands a value back to the caller, whose print displays it. Week 9 will develop returned values and local state further.

def introduces the **definition**. The **function** name is followed by **parameter** names in parentheses and a colon. Four-space indentation marks the body. Executing the **definition** creates the **function** but does not run its body.

The two unindented **call**s then run it, producing 8 and 3 on separate lines. A blank line helps readability; indentation determines the block.

Trace call events

Execution event	What happens
Execute the definition	Create the function; no output yet
Call with 12 and 4	Match parameters; subtract; print 8
Call with 10 and 7	Match new values; subtract; print 3
After the second call	Continue with the next statement

In show_remaining(12, 4), 12 and 4 are **argument**s; capacity and used are **parameter**s.

Positional **argument**s match from left to right. show_remaining(4, 12) produces -8, showing why order matters; it also violates our written **input** contract. Use a fresh file with the **definition** before its **call**s.

Event	Printed output?	Value flow
Execute def	None	Function object created; body not called
show_remaining(12,4)	8 from inside the body	Arguments bind capacity/used; calculation printed
subtract_demo(10,4)	None from that function	return gives 6 to caller

Event	Printed output?	Value flow
print(value) after preview call	6	Caller displays previously returned value

Preview return-to-caller and outer print

```
def subtract_demo(left, right):
    return left - right

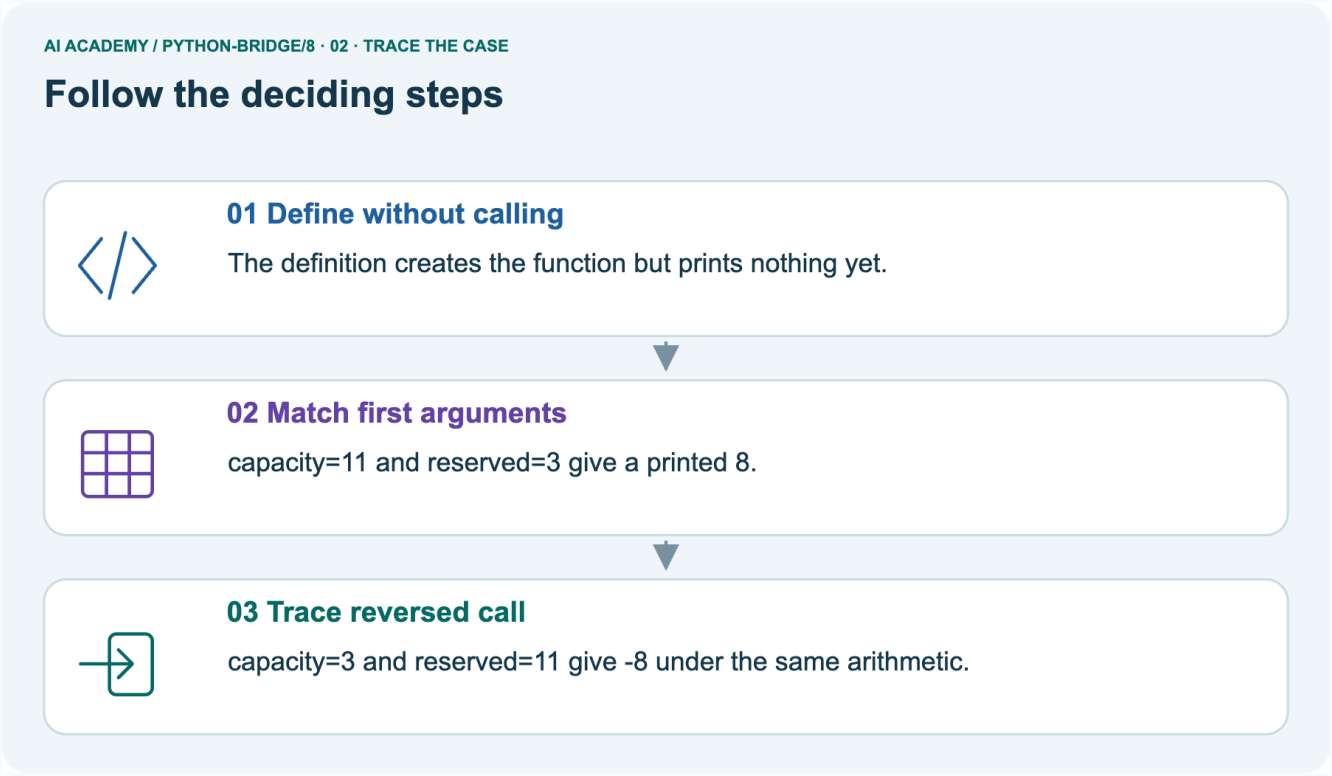
value = subtract_demo(10, 4)
print(value)
```

Read the separate arithmetic reversal

Match **argument**s to **parameter**s. A **function** prints the difference between capacity and reserved spaces.

Python 3: def show_free(capacity,reserved): print(capacity-reserved). **Call**s: show_free(11,3); show_free(3,11).

Trace positional bindings



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Define without calling	The definition creates the function but prints nothing yet.
2. Match first arguments	capacity=11 and reserved=3 give a printed 8.
3. Trace reversed call	capacity=3 and reserved=11 give -8 under the same arithmetic.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Positional **argument**s match **parameter** names in order. A **function definition** is not an execution of its body; separate **call**s supply separate **input** values.

```
def show_free(capacity, reserved):
    print(capacity - reserved)

show_free(11, 3)
show_free(3, 11)
```

Predict definition without calls

Change: Remove both **call**s and leave the **definition** only.

Prediction: No **output** is printed.

Reason: The body runs only when the **function** is **call**ed.

State negative-result domain limits

The arithmetic permits negative results; a real capacity contract may reject reserved>capacity instead of treating it as free space.

Cover the result. Ask learners to name the **input**s and predict the next step. Uncover one step, connect the icon to the actual procedure, and explain its **evidence**. Then change the supplied condition and compare.

The picture is a representation; it does not document execution of a real AI model.

▼ Additional guided key

The **call**s print 0 then 9. capacity receives 9 in both **call**s; used receives 9 then 0. Removing both **call**s leaves a **definition** with no printed **output**. Exit: show_remaining(10, 7) should print 3 but the faulty version prints 8.

Passing two cases does not establish correctness for all allowed **input**s or validate the **input** contract.

▼ Earlier wording, alternatives and picture transcripts

Term	Meaning
Function	A named block of reusable instructions that runs when called
Definition	Code that creates a function and specifies its body
Call	An instruction that runs a function with supplied arguments
Parameter	A name in a function definition that receives an argument
Argument	A value supplied when calling a function

Arguments matched in order

CALL	PARAMETER VALUES	PRINTED OUTPUT
show_remaining(12, 4)	capacity 12; used 4	8
show_remaining(10, 7)	capacity 10; used 7	3

Each call runs the same body with the values supplied for that call.

Model the reasoning aloud

Use the complete worked example below before asking learners to complete W2. Reveal a small part, invite a **prediction**, then explain the deciding step.

These teacher notes contain solutions; do not reveal the independent case answer during modelling.

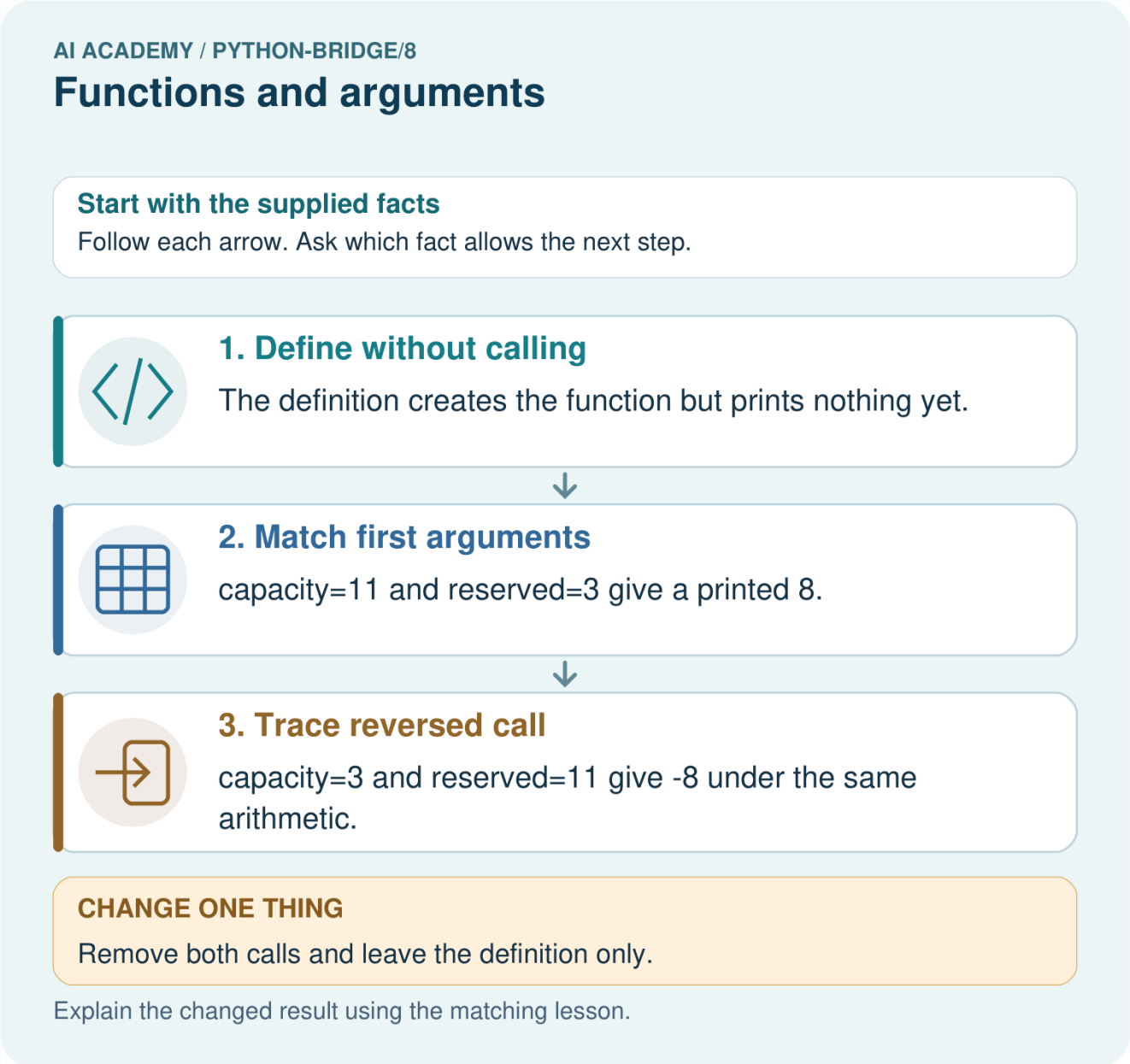
> **Predict the output of show_remaining(9, 9) followed by show_remaining(9, 0). Name the parameter receiving each argument. What would be printed if both calls were removed?**

Thinking move	Teacher prompt
Plan	What is given? What is the question? Which procedure fits these facts?
Do	Point to each input and intermediate step in the worked trace. Name the rule that allows the next step.
Check	Does the output answer the question? Which assumption or limitation prevents a larger claim?

Use the diagram and verbal explanation together. Ask learners to match a sentence to a specific visual step; do not assign learners fixed visual or auditory learning styles.

A pictorial worked case: Functions and arguments

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

Change one thing in the pictured case

Ask while showing the picture

MAIN LESSON · SECTION 3

Guide the trace, then fade the help

> Collect predictions before execution or feedback.

Main completion task and key

Completion task: Using the correct **definition**, predict show_remaining(8, 3), show_remaining(8, 8) and show_remaining(8, 0) in that order. Explain what happens if the file contains only the **definition**.

Teacher check: The **call**s print 5, 0 and 8 on separate lines. A **definition** alone creates the **function** but prints nothing because its body has not been **call**ed.

Start by identifying the given **input**s together. Leave the intermediate steps blank. If needed, model one step, then withdraw that help on the next step. Move to a full trace only when the partial trace is understood.

For partners, alternate explainer and checker; collect a response from every learner before discussion.

Individual misconception check

Use the worked case for **Function**s and **argument**s. Proposed explanation: “Writing def runs the body immediately”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Expected correction: The **definition** creates the **function**; a **call** runs its body.

Collect explanations, not only yes/no votes. If a misunderstanding is common, re-model the relevant step for the class.

If it affects a few learners, give a short targeted group explanation while others solve a more demanding variation.

Pictorial question and changed-case key

Match every **argument** to a **parameter** for the two **call**s. Why do **output**s differ, and what prints when both **call**s are removed?

The first **call** binds 11 to capacity and 3 to reserved, printing 8. The second reverses those bindings and prints -8. A **definition** with no **call**s prints nothing.

Changed-case reasoning: No **output** is printed. The body runs only when the **function** is **call**ed.

Collect each learner's explanation before discussion. Use the first missing connection between **input**, deciding step and result to choose support. This question is worked-case practice, not fresh mastery **evidence**.

▼ Support and alternative plan

Time or stage	Teacher action
Session 1 start to 7 minutes	Retrieve ordered subtraction with the prerequisite.
7 to 20 minutes	Read the contract and label the definition and calls for W1.
20 to 33 minutes	Match argument cards to parameters and trace the worked calls.
33 to 45 minutes	Complete W2 and the guided question before running the file.
Session 2 start to 8 minutes	Retrieve why a definition alone prints nothing.
8 to 20 minutes	Run the fixed-number bug, complete W3 and rerun the earlier case after repair.
20 to 35 minutes	Collect the independent W4 function and predictions before hints.
35 to 45 minutes	Discuss W5 and use the exit to assess the limits of passing tests.

Use the worked example in Sections 3 and 6. Collect a **prediction** before revealing its result; ask the learner to point to each intermediate fact.

Use two labelled **parameter** spaces and place **argument** cards left to right. Permit an oral execution trace alongside code.

Extension: add an if check inside show_remaining that prints Invalid when used exceeds capacity, otherwise prints the difference; test (4, 5) and (4, 4). This checks only that one constraint.

▼ **Earlier wording, alternatives and picture transcripts**

Compact alternative: this two-session sequence covers selected core tasks. Use the three-session weekly plan for the full lesson, independent transfer, feedback and project work; allow extra time for extensions.

Reduce help deliberately

Individual reasoning check

Pictorial practice question

Reasoned teacher answer

MAIN LESSON · SECTION 4

Run the complete supplied investigation

> Preserve expected, observed and paper-trace records.

Supplied case materials and checked guidance

W2 prints 5,0,8; no calls means no output; show_remaining(8) raises missing-argument TypeError. Fixed-constant function on (11,2) prints 8 rather than 9; repair to capacity-used and regression-check another call. Picture outputs 8,-8; definitions only silent. Selected returned differences 7 and -7 are displayed by outer prints; positional binding first/second determines subtraction.

Next week develops return/local state further; main capacity input validation is not implemented.

Diagnose misconceptions

Writing def runs the body immediately: The **definition** creates the **function**; a **call** runs its body.

Arguments and **parameter**s are interchangeable labels: **Parameter**s are names in the **definition**; **argument**s supply values at a **call**.

Swapping **argument**s never matters: Different **parameter** roles can give different results when values are swapped.

A correct first example proves the **function** is reusable: Fixed numbers can match one case while ignoring later **argument**s.

Use Section 9 for the **evidence** ledger. Add a boundary or missing-**input** case and record the expected response before checking. More successful repeats of one case do not establish wider coverage.

Choose accessible support

What the response shows	Next teaching action
Missing or misread facts	Read the case aloud, mark supplied versus unknown facts, and let the learner restate it.
Wrong procedure	Return to the deciding step in the worked case. Contrast it with the proposed incorrect explanation.

What the response shows	Next teaching action
Arithmetic or implementation error	Trace intermediate values or lines; preserve the first mismatch and retry that step.
Correct result without a reason	Ask for a trace and a changed case before marking independent understanding.
An unsupported wider claim	Ask which supplied evidence supports the claim and which further observation would be needed.

Offer an oral explanation, drawing, enlarged visual or fewer examples when that removes a reading/writing barrier. Keep the same reasoning goal.

Stretch secure learners by changing a boundary or assumption and asking them to defend the effect, not simply by assigning more of the same questions.

▼ **Earlier wording, alternatives and picture transcripts**

Match the support to the first error

MAIN LESSON · SECTION 5

Assess the selected bridge transfer task

> If a learner has seen this exact case, use a fresh parallel input and record exposure/support.

Selected independent task and checked key

Selected case: Predict both returned and printed values, showing which **argument** supplies first and second. What is displayed if the program contains only the **function definition** and no **call**s?

Why does swapping these **argument**s change the result?

```
def difference(first, second):
    return first - second
print(difference(11, 4))
print(difference(4, 11))
```

The first **call** returns and prints7; the second returns and prints−7. Positional **argument**s fill first then second, and subtraction is ordered. A **definition** alone displays nothing and does not execute its body.

The same **function** uses the supplied values for each **call**.

Judge trace and support

Dimension	Evidence required
Explanation	Names the applicable idea or procedure and ties it to the supplied facts.
Trace and result	Preserves the given inputs and shows the relevant intermediate steps; distinguishes a prediction from an observed execution.
Limits	States the assumptions, missing evidence or conditions under which the result would change.

Record each dimension as not yet, with support, or independent. Accept a valid alternative procedure with a defensible trace.

Do not penalise an honestly recorded model failure if the learner correctly explains its **evidence** and limits.

Give one repair action

Name the earliest unsupported step and give one action to repair it. Have the learner make that repair now and explain why. Then assess the relevant skill on a fresh, fully specified case.

Confirm it was not previously practised with feedback; solve and check the new case before assigning it. A follow-up design prompt is a starting brief, not automati**call**y an equivalent test.

W1 Label the parts

In def show_remaining(capacity, used), identify the **function** name and **parameter**s. In show_remaining(15, 6), identify the **argument**s and their matching **parameter**s.

Name show_remaining; **parameter**s capacity and used. **Argument**s 15 and 6 match capacity = 15 and used = 6. Expected printed result 9.

W2 Follow the execution order

Using the correct **definition**, predict show_remaining(8, 3), show_remaining(8, 8) and show_remaining(8, 0) in that order. Explain what happens if the file contains only the **definition**.

The **call**s print 5, 0 and 8 on separate lines. A **definition** alone creates the **function** but prints nothing because its body has not been **call**ed.

W3 Repair ignored arguments

The faulty body calculates 12 - 4 regardless of the **call**. State the **output** and expected **output** for show_remaining(11, 2). Repair the expression and name an earlier **call** to rerun.

Faulty **output** 8; expected 9. Use capacity - used. Rerun show_remaining(12, 4), expecting 8, as well as the revealing **call**.

W5 State the input assumption

The correct subtraction **function** prints -1 for show_remaining(4, 5). Does successful execution make this a valid cupboard record? Explain what the **function** assumes and what needs checking.

No. used exceeds capacity and violates the contract. The **function** calculates with supplied values but does not check the contract. A separate validation check or guarded **function** is needed to reject invalid records.

▼ Earlier wording, alternatives and picture transcripts

Guided and exit questions

The **call**s print 0 then 9. capacity receives 9 in both **call**s; used receives 9 then 0. Removing both **call**s leaves a **definition** with no printed **output**. Exit: show_remaining(10, 7) should print 3 but the faulty version prints 8.

Passing two cases does not establish correctness for all allowed **input**s or validate the **input** contract.

W1 Label the parts

W2 Follow the execution order

W3 Repair ignored arguments

W4 Create a new function independently

Each sealed box contains 6 counters. Define show_stock(boxes, loose) to print the total counters using its two **parameter**s. Assume nonnegative integer **argument**s. **Call** it with (3, 2), (0, 2) and (3, 0).

Predict each **output** and explain why show_stock(2, 3) is a different case.

The body prints boxes * 6 + loose. **Output**s are 20, 2 and 18. show_stock(2, 3) gives 15 because boxes and loose have different roles. Define once and use three **call**s; do not hard-code one answer.

W5 State the input assumption

Independent assessment and fresh retry

W4 must define one **function**, use both **parameter**s, **call** it three times and predict 20, 2, 18; the swapped case must give 15.

After support, retry show_tiles(packs, singles) with 4 tiles per pack: **call**s (2, 3), (0, 3), (2, 0) print 11, 3, 8; swapped (3, 2) prints 14. Record assistance and collect new **prediction**s before execution.

Assess independent transfer

Selected case answer

Feedback that the learner uses

MAIN LESSON · SECTION 6

Keep the portfolio, support and delayed check

> Track code execution separately from a correct paper trace.

Project evidence

Add a labelled **function**s and **argument**s **evidence** sheet to Python readiness portfolio. Show how you can distinguish a **function definition** from a **function call**.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Support and selected assessment guidance

Use two labelled **parameter** spaces and place **argument** cards left to right. Permit an oral execution trace alongside code.

Extension: add an if check inside show_remaining that prints Invalid when used exceeds capacity, otherwise prints the difference; test (4, 5) and (4, 4). This checks only that one constraint.

Selected independent answer: The first call returns and prints7; the second returns and prints–7. Positional arguments fill first then second, and subtraction is ordered. A definition alone displays nothing and does not execute its body. The same function uses the supplied values for each call. Keep earlier W4 transfer as additional practice; do not substitute its numbers or key for this selected task.

Record prior exposure and use a fresh parallel case after support.

Delayed return

Suggested return points in this level: python-bridge/9, python-bridge/11. Use actual classroom dates, adjusting for holidays and gaps. One, three and six teaching weeks are scheduling choices, not a claimed optimal spacing.

At a delayed check, ask for the idea from memory and a changed application. Compare with the earlier independent response and record support used.

For a new level, revisit the previous capstone before checking the new prerequisites.

Record	Teacher evidence
Learner code and date	
Concept and case used; prior exposure	
Explanation / trace / limits	
Support and first repair	
Changed case and delayed outcome	
Next teaching action	

Use the **evidence** to choose reteaching or the next step. A completed page, confidence rating or attractive project does not by itself demonstrate understanding.

Bridge to the next topic

Week 9 returns results to the **call**er and explains local variables and fresh state on each **call**.

Retain the project ledger and assess a changed case after support. Check the relevant prerequisite before carrying this component forward.

▼ Earlier wording, alternatives and picture transcripts

Plan the delayed return

Project evidence from this case

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Alternative planning checklist**

REFERENCE · SECTION 7

Reference · Alternative planning checklist

> Use alongside the main route without repeating the teaching cycle.

Planning and assessment reference

Teaching target: Distinguish a **function definition** from a **function call**

Prepare the labelled worked-case picture, the matching workbook, fictional **input** cards and a place for individual responses. Check a local Python 3 environment before class; have a paper-trace version ready.

Session · Plan · Resources

> **Session:** Session 1 · Understand · 45 min
>
> **Plan:** 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.
>
> **Resources:** Sections 1–6; record the brief recall in the workbook.

> **Session:** Session 2 · Practise · 45 min
>
> **Plan:** 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.

> **Resources:** Sections 7–10; use the matching workbook W2 and misconception check.

> **Session:** Session 3 · Investigate and transfer · 45 min
>
> **Plan:** 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.
>
> **Resources:** Use “Run the investigation”, “Connect your project”, “Show what you know”, “Study a complete case” and “Try a new case independently”. Give extra time for the full task set. If this week has a coding lab, use its named section; section numbers can change when extra explanations are included.

Before class, solve the supported practice: Using the correct **definition**, predict show_remaining(8, 3), show_remaining(8, 8) and show_remaining(8, 0) in that order.

Explain what happens if the file contains only the **definition**.

Reasoned practice answer: The **call**s print 5, 0 and 8 on separate lines. A **definition** alone creates the **function** but prints nothing because its body has not been **call**ed.

Check every learner before discussion: Use the worked case for **Function**s and **argument**s. Proposed explanation: “Writing def runs the body immediately”. Decide whether this explanation is supported.

Point to the step or supplied fact that decides; explain your repair if needed.

Misconception repair: The **definition** creates the **function**; a **call** runs its body.

Support: read the exact case aloud, enlarge the labelled picture, supply **input** cards and model only the first deciding step. Accept a spoken explanation or drawing; retain the reasoning goal.

Extension: Remove both **call**s and leave the **definition** only. Require a **prediction** and a reason before comparing. Expected result: No **output** is printed. Reason: The body runs only when the **function** is **call**ed.

Independent assessment: withhold the reserved answer until the learner has attempted the new case. Record support separately from correctness.

Criterion	Not yet	With support	Independent
Explanation	Names an answer without a deciding fact	Uses a hint to link fact and decision	Explains the deciding fact in own words
Trace or test	Cannot connect input and result	Completes steps with prompts	Traces or tests a changed case accurately
Limits and revision	Claims more than evidence supports	Names a limit after a prompt	States a defensible limit and useful next test

Project checkpoint: Python readiness portfolio. Keep runnable code, **prediction**s, actual **output**s, one repaired error and an explanation of a **function** on changed **input**s.

Use the weekly teacher answer for topic-specific reasoning. A saved progress tick is an activity record; judge mastery from the learner's explanation and **evidence**.

▼ **Reference · Feedback and evidence records**

REFERENCE · SECTION 8

Reference · Feedback and evidence records

> Keep individual first attempts separate from supported rehearsal.

Feedback and repair

Use the two formative questions as supported practice, not as independent mastery scores. Discuss the deciding **evidence** rather than accepting a guessed selection.

The feedback comes from the reviewed public worked case. Do not reveal the separately reserved independent teacher answer until the learner has attempted that case.

Ask students to predict before running code. Preserve errors and actual **output**s; collect a changed test and an explanation. The branch-boundary and indexing lessons deliberately include failure cases.

Use the earlier worked-explanation links to address a specific missing building block, then return to the current question. Adapt this suggestion after observing the learner; it is not an automatic diagnosis.

Collect the completed-work portfolio as **evidence**. Record support separately from correctness, and assess explanation, trace/test and limitations. Progress ticks and quiz selections alone do not establish mastery.

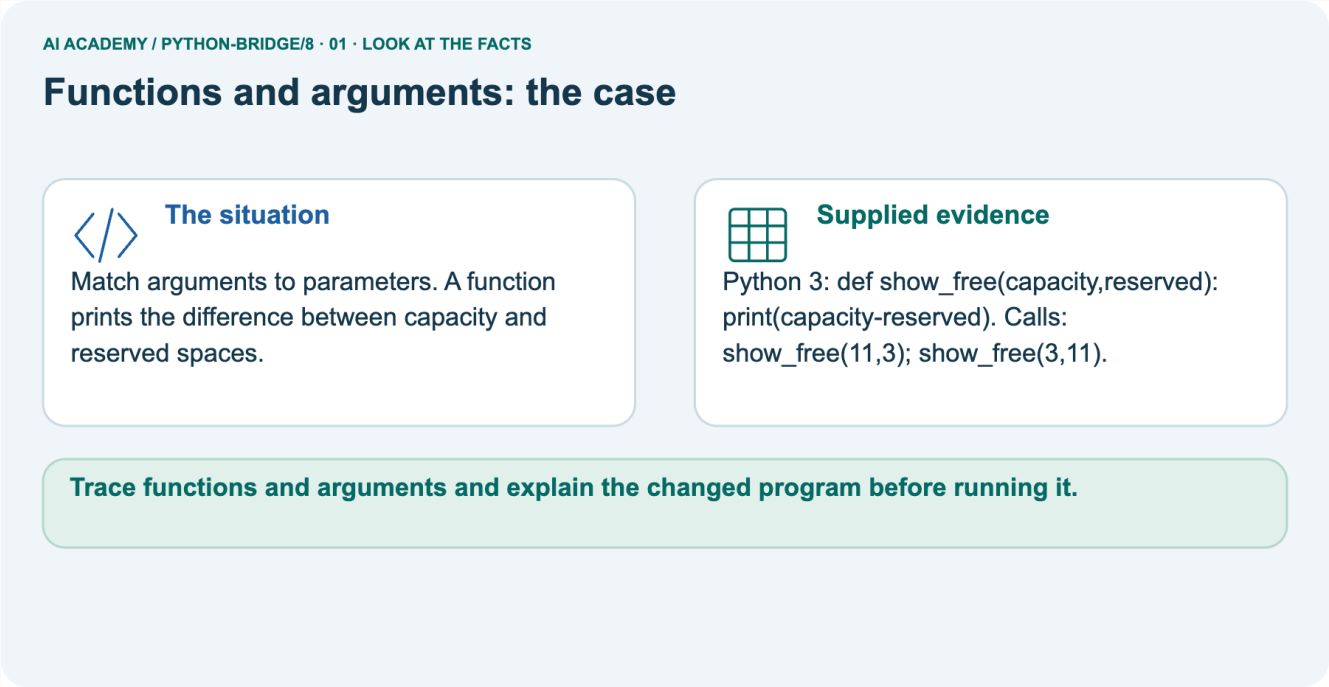
▼ **Reference · Teach with the six pictures**

REFERENCE · SECTION 9

Reference · Teach with the six pictures

> Use the relevant picture for a specific reasoning move.

01. Functions and arguments: the case



Follow the labelled picture with the matching explanation. Open the vector diagram

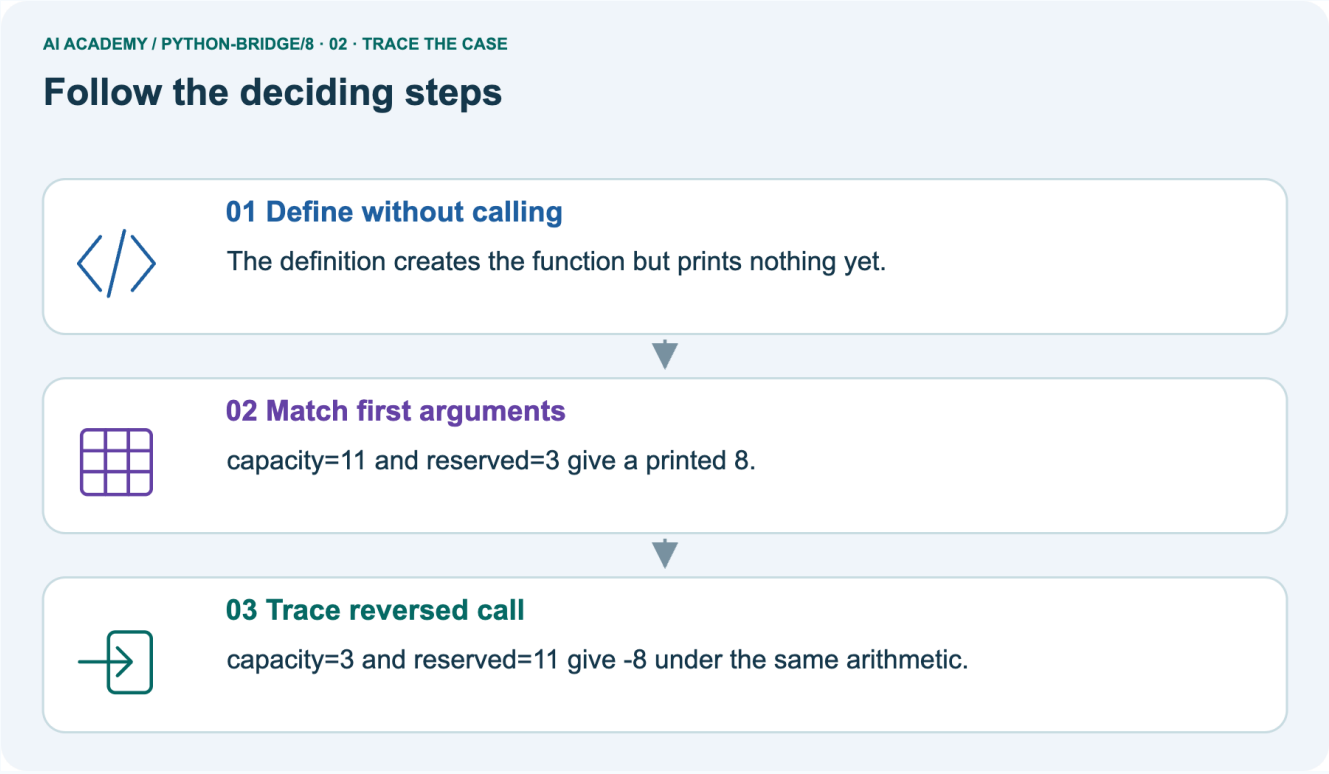
Match **argument**s to **parameter**s. A **function** prints the difference between capacity and reserved spaces. Supplied facts: Python 3: def show_free(capacity,reserved): print(capacity-reserved).

Calls: show_free(11,3); show_free(3,11).

Point to the given fact that matters most. Explain why it matters.

Teacher prompt: Point to the given fact that matters most. Explain why it matters. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

02. Follow the deciding steps



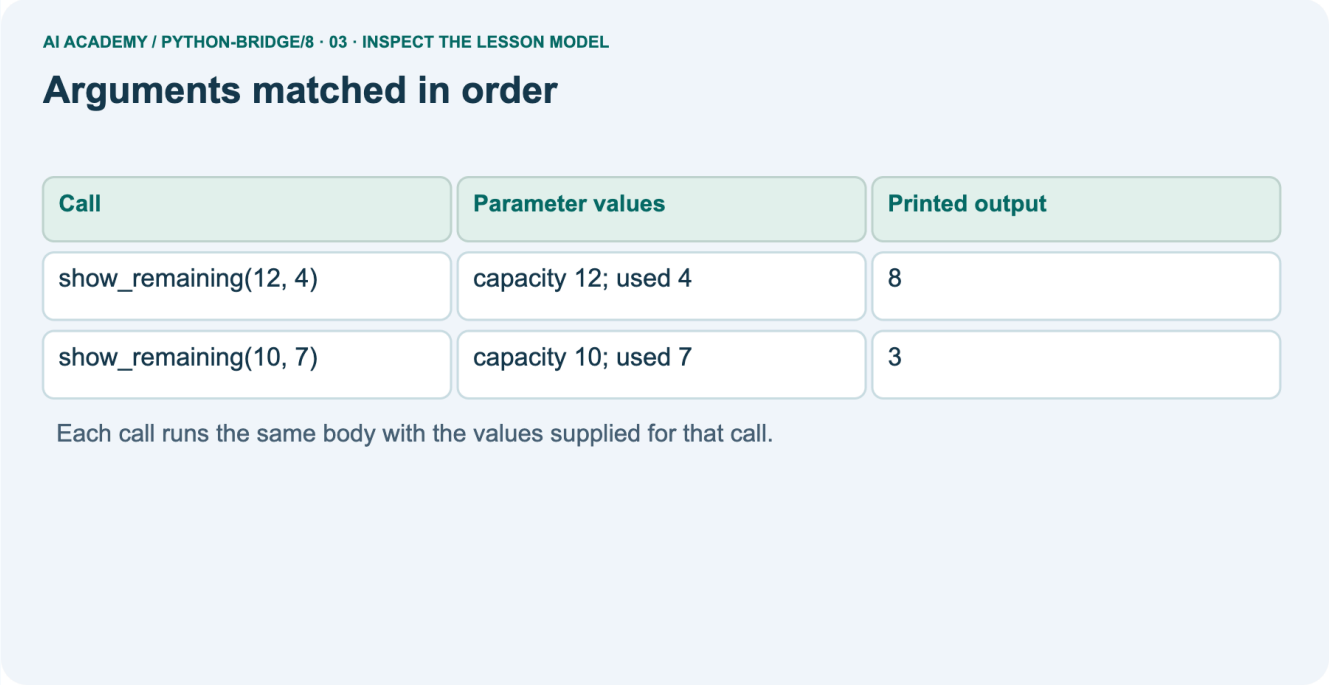
Follow the labelled picture with the matching explanation. Open the vector diagram

Define without **call**ing: The **definition** creates the **function** but prints nothing yet. → Match first **argument**s: capacity=11 and reserved=3 give a printed 8. → Trace reversed **call**: capacity=3 and reserved=11 give -8 under the same arithmetic.

Point to each numbered stage and explain which recorded **input** it uses.

Teacher prompt: Point to each numbered stage and explain which recorded **input** it uses. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

03. Arguments matched in order



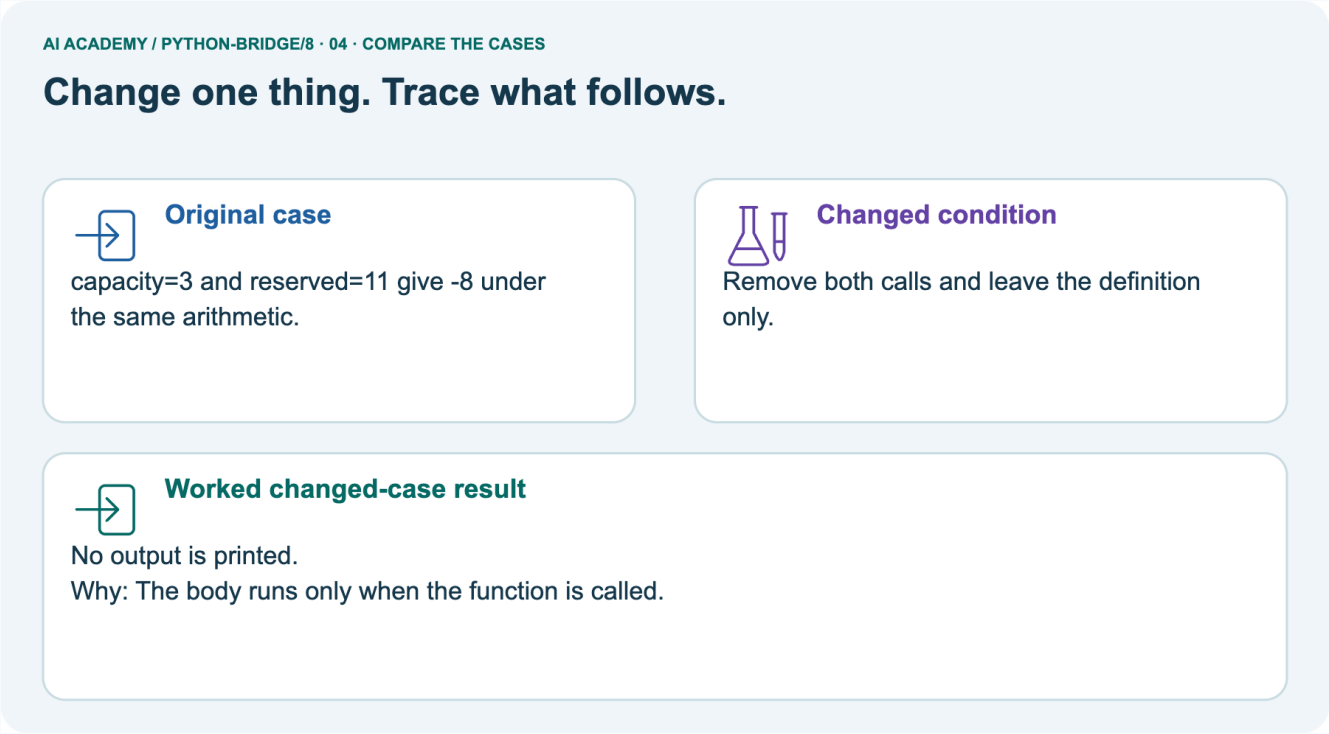
Follow the labelled picture with the matching explanation. Open the vector diagram

Arguments matched in order. show_remaining(12, 4) | capacity 12; used 4 | 8. show_remaining(10, 7) | capacity 10; used 7 | 3

Read the labels and values. Explain the relationship the model shows.

Teacher prompt: Read the labels and values. Explain the relationship the model shows. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

04. Change one thing. Trace what follows.



Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: capacity=3 and reserved=11 give -8 under the same arithmetic. Changed condition: Remove both **call**s and leave the **definition** only. Expected result: No **output** is printed.

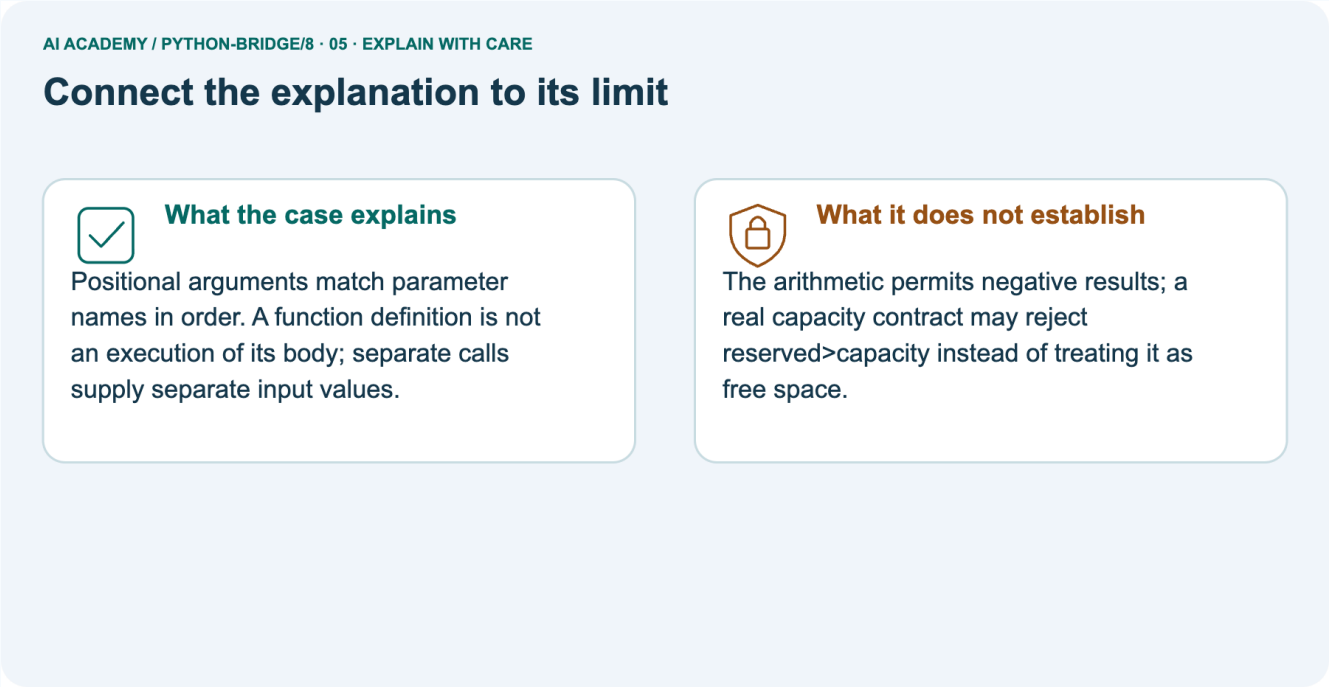
Reason: The body runs only when the **function** is **call**ed.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

Teacher prompt: Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude. Ask learners to describe one label or arrow in ordinary words.

Use the adjacent transcript if the image is difficult to see. Changed-case reasoning: The body runs only when the **function** is **call**ed.

05. Connect the explanation to its limit



Follow the labelled picture with the matching explanation. Open the vector diagram

Positional **argument**s match **parameter** names in order. A **function definition** is not an execution of its body; separate **call**s supply separate **input** values.

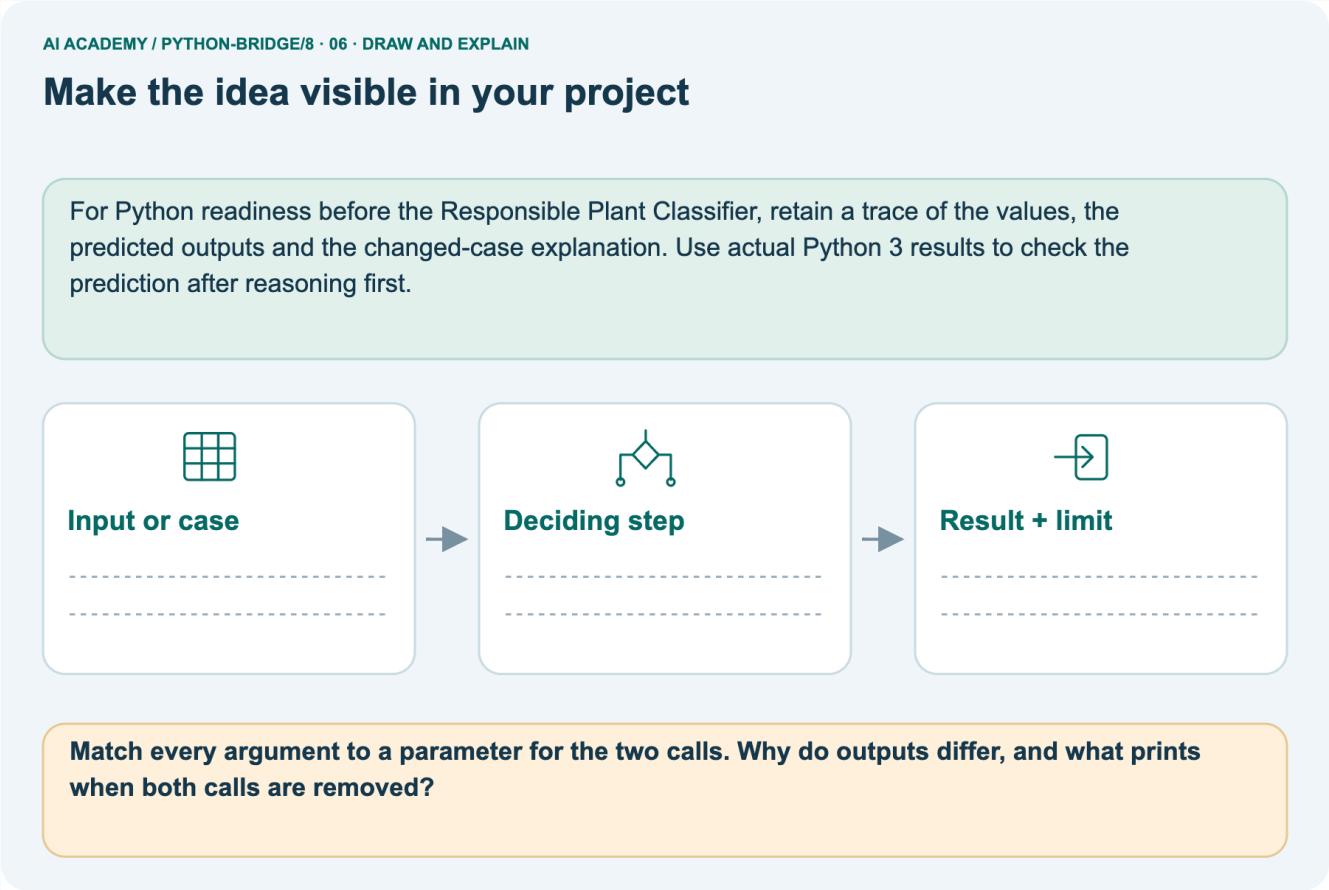
Limit: The arithmetic permits negative results; a real capacity contract may reject reserved>capacity instead of treating it as free space.

Explain the case, then point to a limit on the claim.

Teacher prompt: Explain the case, then point to a limit on the claim. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Keep the stated scope: The arithmetic permits negative results; a real capacity contract may reject reserved>capacity instead of treating it as free space.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Match every **argument** to a **parameter** for the two **call**s. Why do **output**s differ, and what prints when both **call**s are removed?

Draw your project trace and justify the deciding step using **evidence**.

Teacher prompt: Draw your project trace and justify the deciding step using **evidence**. Ask learners to describe one label or arrow in ordinary words. Use the adjacent transcript if the image is difficult to see.

Supported practice key: The first **call** binds 11 to capacity and 3 to reserved, printing 8. The second reverses those bindings and prints -8. A **definition** with no **call**s prints nothing.

▼ Earlier wording, alternatives and picture transcripts

Use one picture at a time. Hide a worked result before asking for a **prediction**, then reveal it for feedback. These are supported teaching cases; retain the separate independent assessment for a fresh first attempt.

01. Functions and arguments: the case

02. Follow the deciding steps

03. Arguments matched in order

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.